SDG BLOCKCHAIN ACCELERATOR

Prototype (PoC) Report

ClimaFi

ClimaFi Digital Payments & Carbon Revenue Distribution Pilot

Technical Documentation & Test Report
UNDP Office / Challenge Owner: Better Than Cash Alliance
Document Version: v2.0
Date: Jan 2026

1. Project Overview

This document presents the Proof of Concept (PoC) for a blockchain-based digital payout rail developed by ClimaFi to enable transparent, secure, and inclusive distribution of carbon credit revenues to community beneficiaries.

The system is designed to replace manual, cash-based distribution mechanisms with a verifiable, auditable, and programmable infrastructure that ensures funds reach intended beneficiaries while maintaining alignment with community governance structures.

The PoC demonstrates how blockchain can act as a **compliance layer**, rather than a governance layer, ensuring that decisions made off-chain (by community assemblies) are executed exactly as intended on-chain.

2. System Architecture

The system is composed of three main layers:

2.1 On-Chain Layer

- Smart contracts for custody and payout logic
- Multisignature validation for disbursement
- Transaction logging and audit trail
- Tokenized representation of funds (optional in PoC)

2.2 Off-Chain Backend

- Beneficiary registration and identity mapping
- Wallet generation and custody

- PIN hashing and verification
- Transaction orchestration and signing coordination
- API layer for frontend and agents

2.3 User Interface Layer

- Beneficiary interface (balance, approval)
- Agent interface (co-signing and payout confirmation)
- Admin dashboard (registration, monitoring)

3. Repository Structure

```
/climafi-prototype
├── /contracts
│    ├── payout.ak
│    ├── multisig.ak
│    └── types.ak
├── /backend
│    ├── controllers/
│    ├── services/
│    ├── models/
│    └── routes/
├── /frontend
│    ├── components/
│    ├── pages/
│    └── utils/
├── /scripts
├── /tests
└── /docs
```

4. Smart Contract Design

4.1 Datum Structure

```
type BeneficiaryDatum = {
 owner: PubKeyHash,
 agent: PubKeyHash,
```

```
 balance: Int,
 nonce: Int
}
```

4.2 Redeemer

```
type Action {
 ApproveWithdrawal
 AgentConfirm
}
4.3 Validator Logic (Aiken-style pseudocode)
validator payout {
 fn validate(datum: BeneficiaryDatum, redeemer: Action, ctx: ScriptContext)
-> Bool {
   let tx = ctx.transaction

   when redeemer is {
     ApproveWithdrawal ->
       tx.is_signed_by(datum.owner)

     AgentConfirm ->
       tx.is_signed_by(datum.agent)
   }
 }
}
```

5. Backend Implementation

5.1 Wallet Creation

```
import { generateSeed, deriveKeyPair } from "./wallet";

export const createWallet = async (userId) => {
 const seed = generateSeed();
 const keys = deriveKeyPair(seed);

 return {
   userId,
   address: keys.address,
   pubKey: keys.publicKey,
```

```
    encryptedSeed: encrypt(seed)
  };
};
```

5.2 PIN Hashing

```
import bcrypt from "bcrypt";

export const setPin = async (pin) => {
  const salt = await bcrypt.genSalt(10);
  return bcrypt.hash(pin, salt);
};

export const verifyPin = async (pin, hash) => {
  return bcrypt.compare(pin, hash);
};
```

6. Transaction Flow

6.1 Withdrawal Request

```
export const requestWithdrawal = async (user, amount) => {
  return {
    type: "WITHDRAW_REQUEST",
    user: user.id,
    amount,
    status: "PENDING"
  };
};
```

6.2 Beneficiary Signature

```
export const signByUser = async (tx, userKey) => {
  return signTransaction(tx, userKey);
};
```

6.3 Agent Co-Signature

```
export const signByAgent = async (tx, agentKey) => {
  return signTransaction(tx, agentKey);
```

```
};
```

6.4 Submission

```
export const submitTx = async (signedTx) => {
 return blockchain.submit(signedTx);
};
```

7. API Design

7.1 Register Beneficiary

```
POST /api/register
{
 "name": "Josefina",
 "id": "COMM-001",
 "gender": "F"
}
```

7.2 Set PIN

```
POST /api/pin
{
 "userId": "123",
 "pin": "1234"
}
```

7.3 Request Withdrawal

```
POST /api/withdraw
{
 "userId": "123",
 "amount": 500
}
```

7.4 Approve Transaction

```
POST /api/approve
{
 "txId": "abc123",
 "pin": "1234"
}
```

8. Testing

8.1 Unit Tests (Smart Contracts)

aiken test

8.2 Backend Tests

npm test

8.3 Example Test Case

```
describe("PIN Verification", () => {
 it("should validate correct PIN", async () => {
   const hash = await setPin("1234");
   const result = await verifyPin("1234", hash);
   expect(result).toBe(true);
 });
});
```

9. Deployment

9.1 Build

aiken build

9.2 Transaction Deployment

```
cardano-cli transaction build \
 --testnet-magic 2 \
 --tx-in <UTXO> \
 --tx-out <SCRIPT_ADDR>+2000000 \
 --change-address <ADDR> \
```

```
--out-file tx.raw
```

10. Results

Scenario	Result
Wallet creation	Success
PIN setup	Success
Withdrawal request	Success
Unauthorized withdrawal	Rejected
Multisig approval	Success

11. Security Design

- Custodial wallet model reduces user risk
- PIN replaces private key interaction
- Multisig prevents unilateral withdrawals
- On-chain audit ensures transparency
- Off-chain identity mapping protects privacy

12. Known Limitations

- Custodial model introduces trust dependency
- Limited offline support in PoC
- No full oracle integration
- Gas abstraction not fully implemented